

Working Paper

AuthR

Authorship Representation: A Protocol Sketch for the Agentic Era

Version 0.1 • Draft for Discussion

April 2026

*We've spent 20 years solving AuthN and AuthZ. The next 10 will be about AuthR
— who actually authored the decision.*

1. Executive summary

Identity and Access Management has historically answered two questions. **AuthN** asks who you are. **AuthZ** asks what you may do. Both were designed for a world where a human sits in front of a terminal, signs in, and directly invokes a system. That world is ending.

Autonomous agents, AI workers, synthetic personas, and verified digital twins now act across time, systems, and teams on behalf of people and organizations. They compose. They re-plan. They chain tools. They run overnight. When something goes wrong — or right — the question that matters is not *who signed in*, nor *what scope did the token carry*. The question is: **who is ultimately responsible for this action?**

This paper proposes **AuthR** — Authorship Representation — as the third pillar of identity, complementing AuthN and AuthZ. Where OAuth's On-Behalf-Of pattern delegates *access*, AuthR delegates *authorship*: it carries grounded responsibility, intent, and lineage across the entire execution graph. It does not replace OAuth, SAML, OIDC, or UMA. It sits above them.

This is an early sketch. It defines the primitives, offers a draft v0.1 record structure, walks through a flagship use case (a CFO approving a wire via a verified digital twin), and ships a working JSON Schema plus a Python reference implementation. The goal is to make the idea concrete enough to argue with.

2. Why AuthR

Naming matters. A new primitive that does not feel native to engineers who already live in AuthN and AuthZ will not be adopted. Several candidates were weighed:

Candidate	Expands to	Strength	Weakness
AuthR	Authorship, Responsibility, Representation	Keeps the pattern. R maps cleanly to <i>responsibility</i> , <i>representation</i> , and <i>root cause</i> .	Slight ambiguity on which R word is canonical.
AuthS	Source	Emphasizes origin.	Softer than authorship. Reads as provenance only.
AuthP	Provenance	Aligned with data governance language.	Breaks the authorship framing.
AuthA	Attribution	Conceptually clean.	Clunky to say and easily confused with AuthN.

AuthR is the recommended name. The ambiguity of R is a feature: authorship, responsibility, and representation are the same idea viewed from three angles. The protocol exists to make all three verifiable.

AuthN → Who are you? *AuthZ* → What can you do? *AuthR* → Who is responsible for what was done?

3. Why OAuth OBO is necessary but not sufficient

OAuth's On-Behalf-Of (OBO) pattern and RFC 8693 Token Exchange solve a real problem: propagating caller context across a service chain while preserving both actor and subject claims. For single-user, short-lived, synchronous request chains, they are the right tool. AuthR is designed to sit on top of them, not replace them.

OBO begins to break in five places that matter for agentic systems:

Break point	Why OBO struggles	What AuthR adds
Multi-step workflows	Tokens do not naturally persist across evolving plans. Each hop becomes a re-interpretation.	A durable authorship record with a correlation id that spans the full execution graph.
Time	OBO assumes short-lived synchronous execution. Agents run long tasks, async actions, and delayed decisions.	Explicit <code>issued_at</code> , <code>expires_at</code> , and <code>stale_after</code> fields with re-anchoring semantics.
Intent drift	OBO carries identity and scope. It does not carry <i>why</i> the action is happening.	Intent is a first-class object, captured in the author's own terms at authorship time.
Agent autonomy	OBO assumes a fixed call path. Agents form evolving execution graphs.	Scope attenuates at every hop. Author is preserved across arbitrary graph shapes.
Accountability	OBO can say who the token represents. It cannot cleanly answer who decided.	Author is separated from Actor. Grounding evidence makes the difference provable.

OBO answers *can this call be made?* AuthR answers *should this action exist, and on whose authority?*

4. The primitives

AuthR defines six primitives. The first four derive directly from the Verified Intelligence pillars (Grounding, Scope, Provenance, Drift) and extend them for identity contexts.

4.1 Author

The **Author** is the real-world referent whose judgment is being executed. It is a **human, a verified digital twin, an organization, or a committee**. It is *not* the agent that runs the code. Separating author from actor is the structural move that makes responsibility provable.

An Author without grounding is a synthetic persona. A grounded Author is verifiable — tied to an HR record, a professional license, a board approval, or a registry entry with a verifier and an evidence digest.

4.2 Actor

The **Actor** is the entity that actually executes the action. Usually an agent, AI worker, synthetic agent, orchestrator, or tool. Actors carry model manifests (signed code and model hashes) and runtime attestation (TEE quotes, SPIFFE SVIDs, or platform attestation). Swapping the model under a stable agent identifier is a known attack vector; binding the actor to a measured manifest closes it.

4.3 Intent

Intent answers *why this action is being taken*, captured in the author's own words at authorship time. It includes a canonical purpose label, a natural-language statement, a risk tier, and whether a human checkpoint is required before irreversible effects. Intent persists across the chain — it is inherited by downstream hops, not reinvented.

4.4 Scope

Scope is the explicit limits on what the Actor may do under this authorship. Actions, resources, monetary caps, time windows, maximum delegation depth. The central invariant: **scope attenuates monotonically**. A sub-agent can receive a narrower scope. It cannot receive a wider one. The AuthR verifier enforces this, not the application.

4.5 Provenance

Provenance is the lineage. It records the ordered chain of prior AuthR records (each referenced by id and depth), a correlation id that ties the whole execution graph together, and the data sources (datasets, policies, prior rulings, retrieved documents) that shaped the decision. Without provenance, authorship collapses into a claim with nothing behind it.

4.6 Drift

Drift is first-class awareness of uncertainty and staleness. Confidence at decision time. A `stale_after` timestamp. Observed deviation signals indicating the original intent may no longer match reality. An AuthR record that is not drift-aware is not really governed — it is hopeful.

5. AuthR v0.1 — draft specification

5.1 Scope of this version

- Defines the **AuthR Record** — a signed, time-bound, auditable object asserting authorship for a unit of agentic work.
- Defines **issuance** (creating a root record anchored to a human author) and **extension** (adding a narrower hop to a chain).
- Defines **verification** invariants: signature, expiry, stable author, monotonic scope attenuation, chain continuity.
- Does **not** yet define the wire format for cross-domain federation, revocation propagation beyond a single control plane, or multi-author records (committee authorship). These are marked as open questions in section 8.

5.2 Record structure

An AuthR Record is a JSON object with the following top-level fields:

Field	Purpose
authr_id	Globally unique URN for this record.
version	Spec version. "0.1" in this draft.
issued_at / expires_at	Issuance and expiry. Short-lived by default.
author	The responsible party. Includes grounding evidence.
actor	The executing agent or worker. Includes model_manifest and attestation.
intent	Purpose, statement, risk tier, HITL flag.
scope	Actions, resources, constraints (amount, time, depth).
provenance	Chain of parent record ids, correlation id, data sources.
drift	Confidence, stale_after, deviation signals.
revocation	Optional endpoint for real-time status checks.
signature	Detached signature over the canonicalized record (minus this field).

5.3 The three operations

5.3.1 Issue (root)

A root AuthR record is minted when a human author (directly or via a verified digital twin that represents them) authorizes an agent to act. The record carries the author, the actor, intent, scope, and any relevant data sources. It is signed by an **Issuing Authority** — a control plane component comparable in role to an OAuth authorization server, but for authorship rather than access.

5.3.2 Extend (hop)

When an orchestrator delegates a sub-task to a sub-agent, it calls `extend` on the parent record. The Issuing Authority produces a new record with:

- The **same author** (authorship does not change hands).
- A **new actor** (the sub-agent).
- An **attenuated scope** (subset of the parent's).
- An **inherited intent** (the parent's statement carries forward).
- A **link to the parent** in `provenance.chain` and the **same correlation id**.

5.3.3 Verify

A relying party verifies a record or full chain against a trust store. Invariants:

1. Signature is valid and the `kid` is trusted.
2. The record is not expired.
3. Across the chain, the `author.id` is stable.
4. Across the chain, `scope` monotonically narrows.
5. Each non-root record's last `provenance.chain` entry references the immediate parent.
6. `correlation_id` is consistent across the chain.

5.4 Relationship to existing standards

Standard	Role	How AuthR relates
OAuth 2.0 / OIDC	Authentication and access token issuance.	Used unchanged. The actor presents OAuth tokens for API access; the AuthR record travels alongside as an additional assertion.
RFC 8693 Token Exchange	Cross-domain delegation of access.	AuthR extension is the authorship analogue. Both can be used together: OBO for the token, AuthR for the decision.
RFC 9396 Rich Authorization Requests	Structured, fine-grained permissions.	AuthR <code>scope.constraints</code> can reference or embed RAR <code>authorization_details</code> .

UMA 2.0	User-managed access for resource sharing.	UMA grants access; AuthR declares who is responsible for the action taken under that access.
SAML 2.0	Federated authentication assertions.	SAML asserts who signed in. AuthR asserts who authored the decision a downstream agent is executing.
W3C Verifiable Credentials / DIDs	Portable identity assertions.	Natural fit for <code>author.id</code> and <code>grounding.evidence_digest</code> .
SPIFFE / SPIRE	Workload identity.	Natural fit for <code>actor.id</code> and attestation evidence.
CoSAI Agentic IAM	Agents as first-class identities.	AuthR is the authorship layer above the agent identity layer CoSAI defines.

6. Flagship use case: CFO approves a wire via a verified digital twin

This is the scenario AuthR was designed for. It is high-stakes, multi-hop, asynchronous, agent-driven, and auditable after the fact — exactly the regime where OBO alone falls short.

6.1 Setting

Acme Corp's CFO, Jane Doe, has a **verified digital twin** authorized to act on her behalf for routine treasury operations up to a governed ceiling. The twin's training data, scope, and revocation controls are all registered in the enterprise agent registry. A treasury orchestrator agent receives instructions through the twin; a wire-validation sub-agent performs preparation checks; the wire service (the final enforcement point) executes the transfer.

6.2 The chain

7. **T0 — Authorship anchors.** The CFO (via her verified digital twin) approves a quarterly supplier payment. The Issuing Authority mints an AuthR root record. Author = Jane Doe (CFO), grounded by her HRMS record and board approval. Actor = treasury orchestrator agent with TEE-attested model manifest. Intent = *release Q2 supplier payment per approved schedule; do not release if variance > 5% or counterparty changed in 48h*. Scope = wire.prepare, wire.validate, wire.approve, wire.submit on a specific account and counterparty, max \$250K, 30-minute window, delegation depth 2.
8. **T1 — Sub-agent hop.** The orchestrator delegates prepare and validate to a narrower validator agent. The Issuing Authority extends the chain: same author, new actor, attenuated scope (only wire.prepare and wire.validate, no approve or submit). Intent carries forward unchanged. Correlation id is preserved.
9. **T2 — Enforcement.** The validator runs its checks and returns findings. The orchestrator prepares the final wire. The wire service — acting as the last-mile enforcement point — receives the root AuthR record plus the hop. It verifies the chain: signature, expiry, stable author, monotonic scope, chain continuity, drift within bounds.
10. **T3 — HITL.** Because `intent.human_in_the_loop = true` and the risk tier is high, the wire service pauses for a live confirmation from the CFO (or a pre-authorized delegate). This check is itself recorded as a child AuthR record.
11. **T4 — Execution and audit.** The wire submits. Every record in the chain is retained in the immutable audit log, indexed by correlation id. A compliance officer can reconstruct, six months later: who authored it, which agent executed which step, what intent was stated, what data sources were consulted, what confidence the system had, and exactly what was permitted at each hop.

6.3 What would have failed under OBO alone

- **Intent drift.** If the orchestrator re-planned and issued a slightly different wire (same API, different rationale), an OBO token would still authorize it. AuthR's intent statement makes the mismatch legible.
- **Long-running execution.** A token minted at T0 and used at T3 an hour later would be suspect. An AuthR record with explicit `stale_after` forces a re-anchor before irreversible effects.

- **Accountability.** An OBO audit answers *which token was used*. An AuthR audit answers *whose judgment was executed* — the CFO's — and proves it with grounding evidence.
- **Scope expansion.** A compromised sub-agent cannot widen its own scope in an AuthR chain. The verifier rejects it before the request reaches the wire service.

7. Sample schema and code

The full JSON Schema and a runnable Python reference implementation are shipped alongside this paper. Abbreviated excerpts follow.

7.1 Example AuthR Record (abbreviated JSON)

```
{
  "authr_id": "urn:authr:01HZXQ6K8Y4F9D3W2P7N5V0A1B",
  "version": "0.1",
  "issued_at": "2026-04-20T14:02:11Z",
  "expires_at": "2026-04-20T14:32:11Z",
  "author": {
    "id": "did:web:acme.com:people:jane-doe",
    "type": "verified_digital_twin",
    "role": "CFO",
    "display_name": "Jane Doe (CFO)",
    "grounding": {
      "referent_type": "verified_human",
      "verified_at": "2026-04-20T14:02:10Z",
      "verifier": "acme-hrms",
      "evidence_digest": "sha256:a3f2..."
    }
  },
  "actor": {
    "id": "spiffe://acme.com/agents/treasury-orchestrator/instance-42",
    "type": "agent",
    "model_manifest": {
      "code_hash": "sha256:c81e...",
      "model_hash": "sha256:9f0a...",
      "model_version": "2026.04",
      "signer_id": "acme-ai-governance"
    },
    "attestation": { "type": "tee_tdx", "evidence_digest": "sha256:...", "verified_at": "..." }
  },
  "intent": {
    "purpose": "approve_wire_transfer",
    "statement": "Release Q2 supplier payment per approved schedule; halt if variance > 5% or counterparty changed in last 48h.",
    "risk_tier": "high",
    "human_in_the_loop": true
  },
  "scope": {
    "actions": ["wire.prepare", "wire.validate", "wire.approve", "wire.submit"],
    "resources": ["account:acme-opex-7788", "counterparty:acme-supplies"],
    "constraints": { "max_amount": 250000.00, "currency": "USD", "max_delegation_depth": 2 }
  },
  "provenance": {
    "chain": [],
    "correlation_id": "corr-7e21...",
    "data_sources": [
      { "source_id": "policy:treasury-payments-v7", "source_hash": "sha256:...",
"source_type": "policy" }
    ]
  },
  "drift": { "confidence": 0.97, "stale_after": "2026-04-20T14:32:11Z", "deviation_signals": [] },
  "signature": { "alg": "EdDSA", "kid": "treasury-twin-key-1", "value": "..." }
```

```
}

```

7.2 Python reference — issuing and extending

The reference implementation (`authr.py`) ships with dataclasses for every primitive, an `IssuingAuthority` class, and a `Verifier`. The core signing path uses Ed25519 and canonical JSON. An abridged excerpt:

```
class IssuingAuthority:
    def issue_root(self, *, author, actor, intent, scope,
                  data_sources=None, correlation_id=None, confidence=0.95):
        record = AuthRRecord(
            authr_id=new_id(),
            version="0.1",
            issued_at=now(),
            expires_at=(datetime.now(timezone.utc) + self.default_ttl).isoformat(),
            author=author, actor=actor, intent=intent, scope=scope,
            provenance=Provenance(chain=[],
                                correlation_id=correlation_id or f"corr-{uuid.uuid4()}",
                                data_sources=data_sources or []),
            drift=Drift(confidence=confidence, stale_after=..., deviation_signals=[]),
        )
        record.signature = self._sign(record)
        return record

    def extend(self, *, parent, actor, attenuated_scope, confidence=0.9):
        _assert_scope_narrows(parent.scope, attenuated_scope) # invariant
        _assert_depth_ok(parent) # invariant
        new_link = ChainLink(authr_id=parent.authr_id,
                             depth=(parent.provenance.chain[-1].depth if
parent.provenance.chain else 0) + 1,
                             issuer=self.kid)
        record = AuthRRecord(
            authr_id=new_id(), version="0.1",
            issued_at=now(), expires_at=parent.expires_at,
            author=parent.author, # author preserved
            actor=actor,
            intent=parent.intent, # intent inherited
            scope=attenuated_scope, # scope narrowed
            provenance=Provenance(
                chain=[*parent.provenance.chain, new_link],
                correlation_id=parent.provenance.correlation_id,
                data_sources=parent.provenance.data_sources,
            ),
            drift=Drift(confidence=confidence, stale_after=parent.expires_at,
deviation_signals=[]),
        )
        record.signature = self._sign(record)
        return record

```

7.3 Python reference — verifying the chain

```
class Verifier:
    def verify_chain(self, chain):
        for record in chain:
            self.verify_record(record) # signature + expiry

        root_author = chain[0].author.id
        for r in chain[1:]:
            if r.author.id != root_author:

```

```

        raise AuthRError("author drift detected across chain")

    for parent, child in zip(chain, chain[1:]):
        _assert_scope_narrows(parent.scope, child.scope)    # monotonic scope

    for i, record in enumerate(chain[1:], start=1):
        if record.provenance.chain[-1].authr_id != chain[i-1].authr_id:
            raise AuthRError(f"chain break at position {i}")
        if record.provenance.correlation_id != chain[0].provenance.correlation_id:
            raise AuthRError("correlation_id mismatch")

```

7.4 Running the flagship scenario

Executing `python authr.py` produces:

```

=====
AuthR v0.1 – flagship scenario: CF0 wire approval
=====

Root record id:      urn:authr:85f9097f-bac4-4a69-89e8-ac27734c96e4
Hop record id:       urn:authr:def93503-365d-4ec4-bc31-a9ead21806c7
Correlation id:      corr-ccb13bd-efee-4cdf-8e2e-a127e8541a41
Author (preserved):  Jane Doe (CF0) (CF0)
Root actions:        ['wire.prepare', 'wire.validate', 'wire.approve', 'wire.submit']
Hop actions:         ['wire.prepare', 'wire.validate']
Chain depth:         1

Chain verified OK.
Scope-widening attempt rejected: scope expansion: actions {'wire.cancel'}

```

The rejection at the bottom is the point. A sub-agent that attempts to add an action its parent never authorized is denied before it can reach a resource. Monotonic scope attenuation is enforced structurally, not hoped for operationally.

8. Open questions

Work that a v0.2 or a standards-track iteration needs to resolve:

- **Cross-domain federation.** When author and actor sit in different trust domains, how do Issuing Authorities federate? OAuth Federation is the likely starting point.
- **Multi-author records.** Committee authorship (joint CFO + General Counsel approval) needs either nested records or a native multi-author primitive.
- **Revocation propagation.** A revoked authorship should cascade through every downstream hop in near real time. Event-driven revocation plus short TTLs gets most of the way, but edge cases remain.
- **Durable intent.** An intent statement captured at T0 is a snapshot. As execution unfolds and conditions change, the system needs a way to detect *intent drift* and force re-anchoring — drift signals are a placeholder for this.
- **Wire format.** JSON with detached JWS is the obvious path. CBOR / COSE deserves evaluation for constrained environments. Privacy-preserving variants (selective disclosure, BBS+) are worth exploring for regulated data.
- **Naming the role.** *Issuing Authority* is borrowed from PKI. *Authorship Server* tracks the OAuth analogy more cleanly. The name should be decided in public.
- **Governed swarms.** When an authorship chain fans out into a swarm rather than a linear sequence, the scope-narrowing invariant still holds per branch — but reconstructing a single defensible answer from many parallel branches needs more thought.

9. Closing

Delegation of access is a solved problem. Delegation of authorship is not. As agents become the primary actors in enterprise systems, the question that will define the next decade of identity is no longer *who is logged in* but *whose judgment is being executed, and can you prove it?*

AuthR is an attempt to give that question a concrete data structure, a small set of invariants, and a name that fits alongside AuthN and AuthZ. It is deliberately narrow in v0.1: one record, three operations, six primitives. That is enough to have a real argument about it — and to build on top of if it turns out to be right.

| *Delegation is how lineage moves. Without lineage, AuthR collapses.*